

## IN2031 Oblig 1 Rapport

### Introduksjon:

I dette prosjektet har vi utviklet et enkelt skriptspråk kalt AeroScript, som tolker og evaluerer matematiske uttrykk. Språket støtter grunnleggende operasjoner som addisjon, subtraksjon, multiplikasjon, negasjon, generering av tilfeldige tall, samt opprettelse av punkter i et todimensjonalt rom. For å håndtere disse funksjonene, har vi implementert flere viktige klasser.

OperationNode er sentral i systemet og brukes til å representere operasjoner som utføres på to uttrykk, inkludert addisjon, subtraksjon, multiplikasjon, negasjon, generering av tilfeldige tall og opprettelse av punkter. Denne klassen evaluerer de ulike operasjonene basert på operasjonstypen som er spesifisert.

For å representere tall har vi klassen NumberNode, som lagrer en numerisk verdi og returnerer denne når uttrykket evalueres. Point-klassen er derimot brukt for å representere punkter med x- og y-koordinater, som kan opprettes ved hjelp av POINT-operasjonen.

Til slutt har vi klassen Interpreter, som fungerer som tolken. Denne klassen traverserer det abstrakte syntakstreet (AST) og evaluerer hvert uttrykk ved å kalle på de relevante nodene. Den støtter parsing av ulike typer uttrykk og håndterer logikken for å evaluere disse.

Disse komponentene jobber sammen for å muliggjøre tolkningen og evalueringen av uttrykk i AeroScript.

### **OperationNode:**

Denne klassen representerer en operasjon som utføres på to uttrykk, og den evaluerer ulike matematiske og logiske operasjoner. OperationNode-klassen er en viktig del av AeroScript-tolkeren. Den håndterer grunnleggende matematiske operasjoner som addisjon, subtraksjon, multiplikasjon, negasjon, samt spesielle operasjoner som tilfeldig tallgenerering og punktskaping. Gjennom evaluate()-metoden evaluerer klassen verdiene for de gitte operasjonene, som muliggjør beregningene i språket AeroScript.

Metoden evaluate():

Dette er hovedmetoden som evaluerer operasjonen. Den tar ingen parametere og returnerer resultatet av operasjonen basert på hvilken type operasjon som er spesifisert.

Metoden bruker en **switch-setning** til å avgjøre hvilken operasjon som skal utføres.

**NumberNode:**

NumberNode er en del av AST i AeroScript og representerer en enkelt numerisk verdi i et uttrykk. Den evaluerer alltid til den samme verdien som ble gitt ved opprettelsen, noe som gjør den til en uforanderlig node. Koden håndterer verdier enkelt og effektivt, uten behov for komplekse operasjoner.

**Point:**

Point-klassen er en enkel datastruktur som holder på to flyttallsverdier som representerer et punkt i et todimensjonalt koordinatsystem. Den gir metoder for å hente ut disse verdiene, og brukes i evalueringen av uttrykk som involverer punkter, som f.eks. i funksjonen POINT i AeroScript.

**Interpreter:**

I dette prosjektet har vi laget Interpreter-klassen, som er selve tolken for AeroScript-språket. Interpreter går gjennom det abstrakte syntakstreet (AST) som blir laget av ANTLR og evaluerer uttrykk som addisjon, subtraksjon, multiplikasjon, negasjon, tilfeldige tall og punkter. Metoden visitExpression tar for seg hver node i AST og bestemmer hva slags operasjon som skal utføres. For eksempel, ved aritmetiske operasjoner eller spesialfunksjoner, oppretter tolken en OperationNode som utfører operasjonen. Hvis uttrykket ikke passer noen av de definerte typene, vil metoden bruke standard logikk for å håndtere det. På denne måten kan tolken håndtere både enkle og mer komplekse uttrykk på en effektiv måte.

**Main:**

Denne Main-klassen gir et godt eksempel på et program som tolker en enkel fil med uttrykk linje for linje. Den bruker en lexer og parser (fra ANTLR) for å omdanne hver linje med tekst til en trestruktur (AST) som deretter evalueres. Dette er en viktig del av hvordan tolkere og kompilatorer fungerer, og koden er designet for å håndtere ulike typer uttrykk som addisjon, multiplikasjon, negasjon, osv

**OperationNodeTest:**

I OperationNodeTest.java har vi laget tester for å sikre at alle operasjoner i OperationNode fungerer som de skal. Vi har testet operasjoner som addisjon, subtraksjon, multiplikasjon, negasjon, generering av tilfeldige tall, og opprettelse av punkter. Testene viser at hver operasjon gir riktig resultat under ulike forhold. For eksempel, testen for PLUS-operasjonen bekrefter at to tall blir lagt sammen korrekt, mens testen for RANDOM sjekker at et tall blir

generert innenfor riktig intervall. Vi har også testet POINT-operasjonen for å sikre at punkter blir opprettet med korrekte koordinater. Alle testene har gått gjennom, noe som bekrefter at operasjonene fungerer som de skal i tolken.

**InterpreterTest:**

InterpreterTest.java inneholder enhetstester for å sjekke at Interpreter-klassen fungerer riktig. Her bruker vi ANTLR til å lage parse-trær fra forskjellige tekstuttrykk og tester tolken ved å evaluere disse trærne. Testene dekker alt fra enkle uttrykk som  $2 + 3$  og  $2 - 3$  til mer komplekse tilfeller som  $--1$  for dobbelt negasjon. Vi sjekker også hvordan tolken håndterer RANDOM-funksjonen, som skal lage tilfeldige tall innenfor et gitt intervall, og POINT-funksjonen, som oppretter punkter med spesifikke koordinater. Testresultatene viser at tolken håndterer alle disse operasjonene og uttrykkene som forventet, uten feil.